

CHAPTER 2

REVIEW OF RELATED LITERATURE

2.1 Metaheuristic Optimization

Optimization refers to the process of identifying the best feasible solution from a set of alternatives subject to defined objectives and constraints. Many real-world problems, including route planning, production scheduling, and machine-learning feature selection, become increasingly difficult as the number of possible solutions grows. Although exact algorithms can guarantee optimality, their computational requirements may become impractical for large and complex problem instances. Researchers therefore commonly use heuristic and metaheuristic methods to obtain high-quality solutions within reasonable computational limits.

Heuristics are procedures designed to solve particular problems efficiently, whereas metaheuristics are broader search frameworks that can be adapted to different optimization problems. Metaheuristics do not ordinarily guarantee that the global optimum will be found. Instead, they guide the search toward promising areas of the solution space without evaluating every possible solution. Their usefulness is especially apparent in combinatorial problems, where the number of feasible configurations may increase exponentially or factorially as the problem becomes larger.

A central concern in metaheuristic design is the balance between exploration and exploitation. Exploration involves examining new or less-visited regions of the solution space, while exploitation involves refining solutions in areas already known to be promising. Excessive exploitation may cause an algorithm to become trapped in a local optimum. Excessive exploration, on the other hand, may cause the search to spend too much time evaluating weak solutions without sufficiently improving the stronger candidates. Effective metaheuristics attempt to balance these behaviors throughout the optimization process (Youssef et al., 2001).

Metaheuristic algorithms may generally be classified as either single-solution or population-based methods. Single-solution methods maintain one current candidate and repeatedly move to a neighboring solution. Simulated Annealing and Tabu Search belong to this category, although they use substantially different mechanisms for escaping local optima. Simulated Annealing relies on temperature-dependent probabilistic acceptance, whereas Tabu Search uses adaptive memory and temporary restrictions on recently performed moves.

Population-based methods maintain several candidate solutions simultaneously. Particle Swarm Optimization is a population-based method in which particles exchange information through personal and collective experience. This population structure allows PSO to examine several

areas of the solution space during the same search. However, rapid information sharing may also cause the particles to converge prematurely when the population follows an inferior leader too quickly (Kennedy & Eberhart, 1995; Sengupta et al., 2019).

The No Free Lunch principle indicates that no optimization algorithm can be universally superior across every possible problem. An algorithm that performs strongly for one problem category may perform poorly for another. Consequently, the practical value of an algorithm depends on the structure of the problem, the representation of candidate solutions, the neighborhood or movement operators, the selected parameter values, the stopping conditions, and the available computational budget. The performance of an algorithm must therefore be assessed within a clearly defined problem and experimental setting rather than generalized from one isolated application.

Simulated Annealing, Tabu Search, and Particle Swarm Optimization were selected for the present study because they represent three substantially different search strategies. Simulated Annealing represents probabilistic single-solution search, Tabu Search represents memory-guided neighborhood search, and Particle Swarm Optimization represents cooperative population-based search. Comparing them across several problem types may provide a clearer understanding of the conditions under which each method performs effectively.

2.2 Simulated Annealing

Simulated Annealing is a stochastic single-solution metaheuristic introduced by Kirkpatrick et al. (1983). It is inspired by physical annealing, in which a material is heated and then cooled gradually so that its particles can settle into a stable, low-energy crystalline structure. In optimization, the physical state of the material corresponds to a candidate solution, energy corresponds to the objective or cost value, and temperature controls the search's willingness to accept inferior solutions.

The algorithm begins with an initial candidate solution and generates a neighboring solution through a problem-specific modification. If the neighbor improves the objective value, it is accepted immediately. A worse neighbor may also be accepted according to a probability determined by the size of the deterioration and the current temperature. A common expression for this rule is $\text{Acceptance probability} = \exp(-\text{cost increase} / \text{temperature})$. When temperature is high, even some substantially worse solutions may be accepted. This allows the search to cross unfavorable regions and leave local optima. As temperature decreases, the probability of accepting worsening moves also decreases, causing the search to shift gradually from broad exploration to focused exploitation.

This mechanism distinguishes Simulated Annealing from conventional hill climbing. A greedy local-search method accepts only improving moves and may terminate when no better neighbor is available. Simulated Annealing can continue beyond that point by permitting temporary deterioration. The ability to accept worse solutions gives the algorithm a chance to reach stronger regions that would otherwise be inaccessible through strictly improving movement.

The cooling schedule is one of the most important components of Simulated Annealing. It defines the initial temperature, the rate at which temperature decreases, the number of candidate moves evaluated at each temperature, the final temperature, and the stopping condition. A common geometric schedule uses $\text{New temperature} = \text{cooling factor} \times \text{current temperature}$. A cooling factor close to one produces slower cooling and prolonged exploration, while a smaller factor causes more rapid cooling and earlier exploitation.

Cooling too quickly may cause premature convergence because the algorithm loses its ability to escape local optima before enough of the solution space has been explored. Cooling too slowly may improve exploration but can require excessive computation. The appropriate balance depends on the structure of the problem, the cost scale, and the neighborhood operator.

Youssef et al. (2001) demonstrated that the numerical scale of the objective function can significantly affect Simulated Annealing. In their VLSI floorplanning experiment, the objective was represented by a fuzzy membership value between zero and one. This caused meaningful deteriorations to retain relatively high acceptance probabilities, even when the search had reached a low temperature. Multiplying the cost difference by an inflation factor substantially improved performance by reducing the probability of accepting large worsening moves. Their findings show that temperature values cannot be selected independently of the objective-function scale. An apparently poor Simulated Annealing result may therefore reflect inadequate calibration rather than an inherent weakness of the method.

Several studies have proposed adaptive cooling methods to reduce parameter sensitivity. Zhan et al. (2016) developed List-Based Simulated Annealing for the Traveling Salesman Problem. Instead of applying a single predetermined cooling equation, the method maintained a priority list of temperature values. At each stage, it selected the largest temperature, evaluated neighboring routes, and generated a replacement temperature based on the worsening moves accepted during that stage. The method cooled rapidly during the early part of the search and more gradually during later refinement.

The list-based approach reduced dependence on a fixed initial temperature and cooling coefficient. Zhan et al. (2016) reported competitive results on Traveling Salesman Problem instances ranging from 48 to 85,900 cities. Their experiments also examined several neighborhood operators. Inversion was the strongest individual operator, insertion generally performed second, and simple swapping performed worst. A hybrid neighborhood that combined inversion, insertion, and swapping produced the strongest overall results. These findings demonstrate that Simulated Annealing performance depends not only on cooling but also on the quality of the neighborhood moves.

Simulated Annealing has also been adapted to multi-objective and evolutionary optimization. Niño et al. (2012) proposed an evolutionary Simulated Annealing-inspired method for bi-objective Traveling Salesman Problems. Their algorithm generated random weights to define different search directions through the objective space. A simplified weighted objective may be expressed as $\text{Weighted objective} = w_1 \times \text{objective 1} + w_2 \times \text{objective 2}$, where the two weights sum to one. The method combined temperature-based acceptance with

mutation, crossover, and non-dominated solution management. It generated substantially larger Pareto sets than its comparison method, although its evaluation had an important limitation because the reference Pareto front was constructed from the outputs of the same algorithms being compared. Consequently, the reported perfect recovery did not independently establish global Pareto optimality.

Hybrid methods have also incorporated Simulated Annealing into population-based algorithms. Mhamdi et al. (2011) combined Particle Swarm Optimization, Simulated Annealing, and Tabu Search for microwave-image reconstruction. Particle Swarm Optimization provided population-level movement, while Simulated Annealing and Tabu Search locally improved different portions of the swarm. The complete three-algorithm hybrid produced lower reconstruction error than standard PSO and the two partial hybrids. The authors attributed the improvement to the combination of global exploration, probabilistic escape, and memory-guided local refinement.

Simulated Annealing has several important advantages. Its basic structure is conceptually simple, it does not require gradient information, and it can be adapted to continuous, nonlinear, discontinuous, and combinatorial objective functions. Its acceptance of worsening moves gives it a direct mechanism for leaving local optima. The neighborhood operator can also be designed according to the structure of the problem, allowing the method to be applied to route optimization, scheduling, VLSI design, network-flow parameter tuning, feature selection, and inverse reconstruction.

However, Simulated Annealing also has important limitations. Its effectiveness is sensitive to the starting temperature, cooling rate, number of transitions, objective-function scale, and stopping rule. Poorly selected parameters may cause premature convergence or excessively long search. Because the method has no explicit memory of previously explored solutions, it may revisit similar regions or spend substantial computation on low-quality candidates. Its stochastic nature also creates variability across runs, making repeated independent trials necessary when assessing its performance.

2.3 Tabu Search

Tabu Search is a memory-based single-solution metaheuristic formally developed by Glover (1989). It extends conventional local search by allowing nonimproving moves and using information from the search history to prevent cycling. A basic Tabu Search procedure begins with an initial solution, generates a set of neighboring candidates, and selects the best admissible neighbor. The selected neighbor does not need to improve the current solution. By accepting the best available nonimproving move, the algorithm can move away from a local optimum and continue exploring the solution space.

Unlike Simulated Annealing, Tabu Search does not rely mainly on probabilistic acceptance. It uses a tabu list to restrict recently performed moves or recently encountered solution attributes. When a move is performed, its reversal or selected characteristics are prohibited for a limited

number of iterations. These restrictions prevent the search from immediately undoing its decisions and repeatedly cycling among the same solutions.

The amount of time a move remains restricted is known as tabu tenure. A tenure that is too short may allow the search to cycle, while a tenure that is too long may prohibit useful moves and force the algorithm unnecessarily far from promising regions. The appropriate tenure depends on the neighborhood structure, problem size, and solution representation.

Watson et al. (2005) analyzed the behavior of Tabu Search on the Job-Shop Scheduling Problem and found that increasing tabu tenure expanded the effective region explored by the search. Larger tenure values substantially increased solution time because the search was forced farther from previously visited areas. The authors recommended selecting the smallest tenure that still prevents cycling and stagnation. Their findings show that stronger restrictions do not automatically produce better search performance.

Tabu restrictions may occasionally prevent an exceptionally strong move. An aspiration criterion allows a tabu move when it satisfies a sufficiently desirable condition. The most common aspiration rule permits a tabu move when it produces a solution better than the best solution found during the search. This prevents memory restrictions from blocking a new global best candidate.

Tabu Search may incorporate several levels of adaptive memory. Short-term memory prevents cycling by recording recently used moves or attributes. Intermediate-term memory supports intensification by identifying features that frequently appear in high-quality solutions. The search can then concentrate on regions containing those characteristics. Long-term memory supports diversification by recording how often particular decisions have been selected. Frequently used features may be penalized so that the algorithm is encouraged to examine less-visited areas.

Glover (1989) also discussed strategic oscillation, in which the search deliberately moves across boundaries such as feasible and infeasible solutions, high and low constraint violations, or constructive and destructive search phases. This allows Tabu Search to examine areas that ordinary local search would avoid. The method is therefore broader than a simple list of forbidden moves. It is an adaptive-memory framework that uses historical information to influence future movement.

Watson et al. (2005) modeled Tabu Search on job-shop scheduling as a biased one-dimensional random walk. Their experiments indicated that the search typically remained only one or two neighborhood moves away from a local optimum. It repeatedly entered a local optimum, accepted a small number of worsening moves, escaped the local basin, and moved toward another local optimum. Their simplified model explained approximately 95% to 97% of the variation in search cost across several scheduling instance categories.

The study also found that problem difficulty was strongly related to the effective width of the region explored rather than the total number of feasible schedules. The initial solution had limited influence on difficult instances unless it began extremely close to an optimum. This

suggests that the search dynamics and memory structures may become more important than starting quality as the problem becomes harder.

Tabu Search has been applied extensively to discrete and combinatorial problems, including job-shop scheduling, flow-shop scheduling, vehicle routing, feature selection, VLSI floorplanning, facility location, production planning, and network optimization. Its explicit memory helps prevent short-term repetition, while best-admissible movement provides strong local exploitation. Aspiration rules and diversification mechanisms also allow the method to escape restrictive search patterns.

Despite these strengths, Tabu Search has several limitations. Its performance depends heavily on the design of the neighborhood, the tabu attributes, the tabu tenure, the candidate-list size, and the aspiration rule. An ineffective neighborhood may fail to produce meaningful improvements even when the memory system is well designed. Evaluating several neighboring candidates at every iteration may also be computationally expensive, especially when each candidate requires a complex simulation or model evaluation. The method often requires substantial problem-specific engineering because the meaning of a move and its reversal differs across routing, scheduling, and feature-selection applications.

2.4 Particle Swarm Optimization

Particle Swarm Optimization was introduced by Kennedy and Eberhart (1995). It is a population-based metaheuristic inspired by collective behavior in bird flocks, fish schools, and other socially interacting groups. Each candidate solution is represented by a particle moving through a multidimensional search space. A particle stores its current position, its current velocity, the best position it has personally discovered, and information about the best position found by the swarm.

The particle's personal best is commonly called **pbest**, while the best position found by the entire population is called **gbest**. The basic movement principle may be expressed as **New velocity = previous velocity + personal attraction + social attraction**, followed by **New position = current position + new velocity**. The personal component encourages particles to use their own successful experience, while the social component encourages them to learn from the strongest solution discovered by the group.

Momentum and personal attraction support exploration because particles retain their previous movement and may return to locations they individually found useful. Social attraction supports exploitation because the population moves toward the strongest known solution. If social influence becomes too strong, the swarm may converge prematurely around a local optimum. If personal influence becomes too strong, particles may move too independently and fail to benefit from collective information. Kennedy and Eberhart (1995) observed that a reasonable balance between personal and social influence produced the most effective search behavior.

Later PSO developments introduced an inertia weight that controls how strongly particles preserve their previous velocity. A high inertia weight encourages broader movement, while a lower value promotes refinement around known solutions. Many implementations gradually reduce inertia during the search so that the swarm begins with stronger exploration and ends with stronger exploitation. Acceleration coefficients determine the strength of attraction toward personal and social best positions. Velocity clamping and constriction factors may also be used to prevent particles from moving excessively far or oscillating uncontrollably.

Sengupta et al. (2019) emphasized that no PSO parameter configuration is universally optimal. Performance depends on the objective function, dimensionality, topology, population size, and diversity of the swarm. Parameter values should therefore be selected according to the problem and validated experimentally rather than treated as universal constants.

PSO performance is also affected by the swarm topology. In global-best topology, every particle follows the best solution discovered by the entire population. This spreads information quickly but increases the risk of premature convergence. In local-best topology, particles learn only from smaller neighborhoods. This slows information flow but may preserve greater diversity and delay convergence around an inferior leader. Alternative topologies include rings, stars, grids, and randomly changing neighborhoods.

Standard PSO was developed for continuous optimization. Route planning, scheduling, and feature selection are discrete or combinatorial, so the concepts of position and velocity must be adapted. Discrete PSO methods may use binary encoding, sigmoid-based probability mapping, set-based movement, permutation crossover, mutation, rounding, or problem-specific decoding.

Alharkan et al. (2020) used Geometric Particle Swarm Optimization for a scheduling problem involving two parallel machines and one shared setup server. Instead of applying a continuous velocity equation, the algorithm generated a new job permutation through a three-parent crossover involving the particle's current position, its personal best, and the global best. Swap mutation was then used to preserve exploration.

Huang et al. (2016) developed a modified discrete PSO for multi-objective flexible job-shop scheduling. Their representation contained an operation-sequence vector and a machine-assignment vector. The algorithm maintained archives of non-dominated solutions and used Improved Precedence Operation Crossover, Multipoint Preservative Crossover, machine-assignment disturbance, and variable neighborhood search. These mechanisms allowed the algorithm to maintain valid schedules while searching for trade-offs among makespan, total workload, and maximum machine workload.

PSO has also been adapted to feature selection. Zhang et al. (2017) represented each feature through a continuous probability value and selected the feature when the value exceeded a threshold. Xie et al. (2021) used enhanced continuous PSO models whose positions were decoded into binary feature subsets. These studies show that discrete applications often preserve the social-learning principle of PSO while replacing its original movement equation with problem-specific operators.

Multi-objective PSO maintains several trade-off solutions rather than a single global best. It commonly uses an external archive of non-dominated solutions. Crowding distance or similar diversity measures may be used to select guides from sparsely represented areas of the Pareto front. Zhang et al. (2017) used an external archive for multi-label feature selection, where the objectives were to minimize Hamming loss and minimize the number of selected features. Adaptive mutation and local differential learning were used to improve exploration and Pareto-front coverage.

PSO is frequently hybridized with other optimization methods to address premature convergence and weak local refinement. Hybrid approaches may combine PSO with Genetic Algorithms, Differential Evolution, Simulated Annealing, Tabu Search, Ant Colony Optimization, Artificial Bee Colony algorithms, or neighborhood search. Sengupta et al. (2019) identified hybridization as one of the most active areas of PSO research. However, they also noted that hybrid methods often contain many interacting mechanisms and parameters, making it difficult to determine which component produced the observed improvement.

Particle Swarm Optimization offers several advantages. It performs population-based exploration, shares information rapidly, can be implemented using relatively simple operations, and is compatible with parallel processing. Its population allows several areas of the search space to be examined simultaneously. However, it is vulnerable to premature convergence, stagnation, and loss of diversity. Its behavior depends on the quality of the leaders and the selected movement parameters. Discrete problems also require specialized representations and operators that may substantially change the behavior of the original algorithm.

2.5 Benchmark Optimization Problems

The present study focuses on three major categories of optimization problems: route optimization, job scheduling, and machine-learning feature selection. These categories were selected because they represent substantially different solution structures and evaluation requirements. Route optimization commonly uses permutation-based solutions, scheduling combines sequencing and assignment under resource constraints, and feature selection uses binary subsets whose evaluation may require repeated model training.

Using several problem categories may provide a broader understanding of the algorithms than testing them on one application alone. An algorithm that performs effectively in a continuous or permutation-based space may behave differently in a binary subset space. Differences in solution representation, neighborhood structure, objective cost, and constraint handling may favor different search strategies.

2.6 Route Optimization

Route optimization seeks an efficient path among a set of locations while satisfying defined travel or operational constraints. One of the most widely studied route problems is the Traveling

Salesman Problem. It asks for the shortest closed route that visits every city exactly once and returns to the starting point. A candidate solution is commonly represented as a permutation of the cities, and its objective value is the total distance between consecutive cities, including the return to the starting location.

The number of possible routes increases factorially with the number of cities. Consequently, exhaustive enumeration becomes impractical for large instances. This has made the Traveling Salesman Problem a common benchmark for evaluating exact algorithms, heuristics, and metaheuristics.

Grabusts et al. (2019) applied Simulated Annealing to an eight-location route involving dairy enterprises in Belarus. A 2-opt neighborhood generated new solutions by reversing a selected segment of the route. The reported route stabilized at approximately 648.69 kilometers. The study demonstrated a practical implementation of SA for route construction, but the small instance size limited the strength of its conclusions. The result was not compared with an exact optimum, only one principal stochastic outcome was reported, and the temperature and transition parameters were not documented in sufficient detail.

Zhan et al. (2016) provided stronger evidence through List-Based Simulated Annealing. Their method was tested on Traveling Salesman Problem instances ranging from dozens to tens of thousands of cities. The list-based temperature mechanism reduced dependence on manually selected cooling schedules, while a hybrid neighborhood combined inversion, insertion, and swapping. Their results showed that Simulated Annealing could remain competitive on very large routing instances when the cooling method and neighborhood design were carefully constructed.

Tabu Search has also been widely applied to route optimization because route changes can be represented naturally through edge exchanges, segment reversals, city insertion, and route swaps. A tabu list may store recently added or removed edges, preventing the algorithm from immediately reversing a route modification. Pirim et al. (2008) reviewed several routing studies and found that Tabu Search frequently produced stronger solution quality than Simulated Annealing and threshold-accepting methods, particularly as problem size increased. However, Tabu Search sometimes required more computation because it evaluated several route candidates at each iteration.

Ru (2024) applied Tabu Search to multimodal vehicle-logistics routing involving road, rail, and water transport. The model considered transfer costs, transportation time, facility capacity, and route profitability. The company case reported increased profit on most optimized routes. However, the paper contained inconsistencies in its runtime reporting and used classification measures such as recall and ROC-curve area in a routing context without sufficiently explaining their relevance.

Particle Swarm Optimization requires substantial adaptation for route problems because a route is a permutation rather than a continuous vector. Discrete PSO routing methods may represent movement through swap sequences, edge sets, crossover operations, route decoding, or hybrid

local search. Sengupta et al. (2019) reviewed several PSO hybrids for route optimization, many of which combined PSO's population search with 2-opt, 3-opt, Ant Colony Optimization, or Simulated Annealing. These combinations allow PSO to examine several route regions simultaneously while relying on local operators to refine individual routes.

The route-optimization literature suggests that Simulated Annealing offers a simple and flexible neighborhood search, Tabu Search provides strong local exploitation and cycling prevention, and Particle Swarm Optimization offers population-level exploration but requires careful discrete encoding. Hybrid local search often strengthens all three methods. A fair comparison should therefore measure route quality, objective evaluations, runtime, convergence speed, and consistency across repeated runs.

2.7 Job Scheduling and Machine Allocation

Job scheduling assigns operations to machines and determines their processing order. Common objectives include minimizing makespan, minimizing total completion time, reducing tardiness, balancing workloads, and improving machine utilization. Scheduling becomes increasingly difficult when it includes multiple machines, alternative machine assignments, setup times, shared servers, precedence constraints, due dates, or limited capacity.

In the classical Job-Shop Scheduling Problem, each job contains several operations that must be processed on specified machines in a predetermined order. The objective is commonly to minimize makespan, which is the completion time of the final operation. Watson et al. (2005) showed that the search space contains many local and near-local-optimal schedules. Structured workflow and flow-shop instances were particularly difficult because the effective region that Tabu Search had to explore was wider.

The Flexible Job-Shop Scheduling Problem extends the classical formulation by allowing an operation to be processed on one of several eligible machines. The scheduler must decide both the machine assignment and the operation sequence. Huang et al. (2016) treated this as a multi-objective problem and minimized makespan, total machine workload, and maximum individual machine workload. Their modified discrete PSO maintained non-dominated schedules and produced strong, diverse Pareto fronts on the Kacem and Brandimarte benchmark instances.

Alharkan et al. (2020) studied a scheduling problem involving two identical parallel machines served by one setup server. Each job required a setup period and a processing period. Since only one server could perform setup at a time, the algorithm had to coordinate server availability with machine availability. The study compared Tabu Search, Geometric Particle Swarm Optimization, Simulated Annealing, a Genetic Algorithm, and Iterated Local Search. Tabu Search produced the lowest worst-case average error ratio and most frequently reached the theoretical lower bound for large instances. GPSO generally ranked second, while Simulated Annealing and the Genetic Algorithm performed strongly on small instances but deteriorated as problem size increased.

Jwo et al. (2023) applied Tabu Search to manufacturing-order sequencing in an aircraft-industry work center. Their model considered daily capacity, due dates, processing times, remaining lead time, and unfinished orders carried into subsequent days. The neighborhood swapped delayed manufacturing orders with other orders. When initialized using First In, First Out, the method reached the theoretical lower bound for the number of delayed orders on all five tested days. Its total expected delay also remained close to the ideal reference values.

Simulated Annealing represents a schedule as one current sequence or assignment. Neighboring schedules may be generated by swapping jobs, inserting an operation, reversing part of a sequence, or changing a machine assignment. Its probabilistic acceptance rule allows the search to leave locally optimal schedules. However, difficult scheduling instances may require many evaluations before the cooling schedule produces sufficiently focused refinement. Alharkan et al. (2020) found that Simulated Annealing reached the lower bound in all eight-job cases but did not maintain the same performance as the number of jobs increased.

Tabu Search is especially common in scheduling because job exchanges, operation movements, and machine reassignments naturally define neighborhoods. Tabu attributes may include recently swapped jobs, recently changed machine assignments, or recently reversed precedence relationships. The literature indicates that Tabu Search performs strongly when the neighborhood produces meaningful schedule changes, candidate evaluation remains computationally manageable, and tabu tenure prevents cycling without overrestricting the search.

Particle Swarm Optimization must be modified for scheduling because continuous particle movement does not naturally preserve valid job permutations or operation precedence. Discrete PSO scheduling methods replace velocity movement with crossover, swaps, probability vectors, and decoding procedures. Their population-based nature allows several schedule regions to be examined simultaneously, but maintaining validity requires more complex operators than standard PSO.

The scheduling literature generally indicates that Tabu Search performs strongly on medium and large combinatorial instances. Discrete PSO variants can also scale effectively when their representation preserves feasible schedules and maintains diversity. Simulated Annealing may perform well on small or moderately sized instances but can require longer computation on difficult problems. These conclusions remain dependent on the scheduling formulation, neighborhood operators, evaluation limits, and parameter settings used by each study.

2.8 Machine-Learning Feature Selection

Feature selection is the process of identifying a subset of relevant variables from a larger feature set. For a dataset containing n features, there are 2^n possible subsets. Even a moderate number of variables therefore creates a very large combinatorial search space.

The objectives of feature selection include reducing dimensionality, removing irrelevant or redundant information, lowering training cost, reducing storage requirements, improving interpretability, and preserving or improving predictive performance. Removing unnecessary features may also reduce overfitting, particularly when the number of available training samples is small relative to the number of variables.

Feature-selection methods are commonly classified as filter, wrapper, or embedded approaches. Filter methods evaluate features through statistical criteria independently of the predictive model. They are computationally efficient but may overlook interactions among variables. Wrapper methods evaluate a candidate subset by training and testing a classifier or ranking model. They can account for feature interactions but are computationally expensive because each candidate subset may require a complete model evaluation. Embedded methods perform selection as part of the learning process itself.

Allvi et al. (2020) applied Simulated Annealing to feature selection for learning-to-rank systems. A state represented a feature subset of fixed size, while a neighboring state was created by replacing selected feature indices. LambdaMART was used as the ranking algorithm, and Normalized Discounted Cumulative Gain served as the state-quality measure. The search was repeated for each possible subset size.

The authors reported that subsets containing approximately 18 to 39 features could match or exceed the performance of the complete feature sets across six benchmark datasets. Their findings suggested that some original features were redundant or harmful. However, the study used random feature selection as its main baseline, did not report sufficiently detailed repeated-run statistics, omitted several important Simulated Annealing parameters, and appeared to use test performance during feature selection. The last issue creates a risk of test-set leakage and overly optimistic evaluation.

Zhang and Sun (2002) applied Tabu Search to both fixed-size and minimum-size feature selection. For variable-size selection, a neighbor was generated by adding or removing one feature. For fixed-size selection, one selected feature was removed while one unselected feature was added, preserving the required subset size. The method was compared with Sequential Forward Selection, Sequential Backward Selection, floating selection, Genetic Algorithms, and Branch and Bound.

In synthetic experiments involving 30, 60, and 100 features, Tabu Search generally produced better best, mean, and worst objective values than the Genetic Algorithm under similar or lower evaluation costs. In a 20-feature fixed-size task, Tabu Search reached the Branch-and-Bound optimum in all 20 runs when selecting 10 features. The authors also found that tabu tenure affected search quality, larger candidate sets generally improved average performance, simple neighborhoods were more effective than very large neighborhoods sampled too sparsely, and the initial solution had limited influence on the final result.

Xie et al. (2021) developed two enhanced PSO methods, PSOVA1 and PSOVA2, for wrapper-based feature selection. Their evaluation used a K-Nearest Neighbor classifier and

combined classification accuracy with feature reduction. Accuracy received substantially greater weight than subset size. PSOVA1 included Gaussian leader enhancement, mutation of weak particles, strengthened personal and global signals, and spiral local search. PSOVA2 added adaptive exemplar breeding, nonlinear movement coefficients, redistribution of weak particles, and exponential leader search.

Across 13 datasets containing from 30 to 22,283 features, the proposed methods achieved the highest reported accuracy and F-score among the compared algorithms. PSOVA2 demonstrated statistically significant superiority over all baseline methods on eight datasets. However, the proposed algorithms were highly complex. Their improvements resulted from PSO combined with several additional mechanisms, making it difficult to determine how much of the benefit came from the basic swarm structure itself.

Zhang et al. (2017) treated multi-label feature selection as a two-objective problem. The algorithm simultaneously minimized Hamming loss and the number of selected features. It maintained an external archive of non-dominated solutions, used crowding distance to preserve diversity, and applied adaptive mutation and local differential learning. Across six multi-label datasets, the method achieved higher hypervolume than NSGA-II and generally dominated the mutual-information comparison method. The study demonstrated that feature selection does not always need to be converted into one weighted objective. Multi-objective optimization can preserve several useful trade-offs between predictive performance and dimensionality.

The feature-selection literature indicates that all three algorithms can search large subset spaces. Simulated Annealing provides a simple mechanism for moving among subsets but requires careful cooling and repeated model evaluations. Tabu Search provides explicit memory and strong neighborhood control. Particle Swarm Optimization can examine several subsets simultaneously and is well suited to multi-objective archives, but it is vulnerable to diversity loss and requires binary or probability-based encoding.

A fair comparison of feature-selection algorithms should evaluate predictive performance, subset size, runtime, objective evaluations, convergence, and stability across repeated trials. The same classifier, preprocessing method, training-validation-test partition, and performance metric should be used for all algorithms. Candidate subsets should be selected using training or validation data, while test data should be reserved for final evaluation.

2.9 Comparative Studies of Simulated Annealing, Tabu Search, and Particle Swarm Optimization

Several reviewed studies directly compared two or more of the selected metaheuristics. These comparisons provide evidence regarding solution quality, convergence, computational efficiency, and search behavior, although their findings remain dependent on the problem and experimental conditions.

Zhang and Nicholson (2018) compared Boltzmann Simulated Annealing, Very Fast Annealing, Tabu Search, and Particle Swarm Optimization for tuning a parameter in the Parameterized Dynamic Slope Scaling Procedure for Fixed-Charge Network Flow. The study included 180 generated network instances. All four methods improved approximately 94% to 95% of the instances, with average improvements of around 10% and maximum improvements near 24%.

No method was statistically superior in final solution quality. The principal difference concerned efficiency. Particle Swarm Optimization produced the greatest improvement per iteration and was fastest on small and large instances. Tabu Search remained competitive on the largest instances, while both Simulated Annealing variants required substantially more computation. The study demonstrates that similar final objective values may conceal meaningful differences in convergence and evaluation efficiency.

Youssef et al. (2001) compared a Genetic Algorithm, Simulated Annealing, and Tabu Search on VLSI floorplanning under a shared budget of 5,000 objective evaluations. The objective combined circuit area, wire length, and delay through a fuzzy membership score. Across five circuits, the reported ranking was Tabu Search first, Genetic Algorithm second, and Simulated Annealing third.

Tabu Search reached strong solution regions quickly and concentrated most of its evaluations on high-quality floorplans. The Genetic Algorithm improved rapidly during the early generations but later reached a plateau. Simulated Annealing spent more evaluations in weak regions and continued accepting relatively large worsening moves. However, the cost-inflation experiment showed that its poor performance was partly caused by a mismatch between the fuzzy objective scale and the temperature schedule. This emphasizes that algorithm comparisons must consider whether each method has been calibrated appropriately.

Alharkan et al. (2020) compared Tabu Search, Geometric Particle Swarm Optimization, Simulated Annealing, a Genetic Algorithm, and Iterated Local Search on the two-machine single-server scheduling problem. Tabu Search produced the lowest worst-case average error and most frequently reached the theoretical lower bound on large instances. Geometric Particle Swarm Optimization ranked second overall. Simulated Annealing and the Genetic Algorithm were competitive for smaller instances but deteriorated as the number of jobs increased.

Pirim et al. (2008) reviewed many studies comparing Tabu Search with Simulated Annealing, Genetic Algorithms, Particle Swarm Optimization, and Ant Colony Optimization. Their findings indicated that Tabu Search frequently performed strongly in scheduling, routing, facility location, and production planning. However, they also identified cases in which Simulated Annealing produced better schedules or Genetic Algorithms performed better because their representation reduced the number of candidate solutions. Algorithm rankings also changed according to whether comparisons were controlled by runtime, evaluated solutions, or unrestricted convergence. The review therefore supports a problem-dependent rather than universal interpretation of performance.

Mhamdi et al. (2011) compared standard PSO with several hybrid forms in microwave imaging. In one experiment, standard PSO produced a reconstruction error of approximately 0.105, PSO combined with Tabu Search produced approximately 0.047, PSO combined with Simulated Annealing produced approximately 0.024, and the full PSO-SA-TS hybrid produced approximately 0.003. The complete hybrid also reduced computation time in another experiment.

The authors attributed the improvement to the combination of Particle Swarm Optimization's population-level exploration, Simulated Annealing's probabilistic local escape, and Tabu Search's memory-based refinement. However, the study did not compare full standalone implementations of SA and TS under the same reconstruction problem, so its findings primarily support the value of hybridization rather than a direct ranking of the three independent methods.

The comparative literature shows that no one algorithm consistently dominates all problem categories. Tabu Search frequently performs strongly on neighborhood-intensive combinatorial problems. Particle Swarm Optimization often demonstrates strong efficiency and scalability when an appropriate representation is available. Simulated Annealing can produce competitive final quality but may require more evaluations and careful calibration of temperature and objective scale. Hybrid methods may combine complementary strengths, although they also increase implementation complexity and parameter interactions.

2.10 Evaluation Criteria Used in Previous Studies

The reviewed studies used different evaluation measures depending on the optimization problem. Solution quality was commonly assessed through the best, mean, and worst objective values; percentage deviation from a known best solution; error relative to a theoretical lower bound; and success frequency in reaching a known optimum. Reporting only the best result is insufficient because it does not show how consistently a stochastic algorithm performs.

Computational efficiency was measured through CPU time, number of iterations, number of objective-function evaluations, improvement per iteration, and convergence speed. Raw iteration counts should be interpreted cautiously because the meaning of one iteration differs among the algorithms. One Tabu Search iteration may evaluate several candidates, one Simulated Annealing transition may evaluate only one neighbor, and one PSO iteration evaluates the complete swarm. Equal numbers of objective evaluations or equal runtime budgets may therefore provide a fairer comparison than equal iteration counts.

Stability should be evaluated through repeated independent runs. Useful measures include the mean, standard deviation, median, best and worst outcomes, and success frequency. Repeated trials are particularly important for Simulated Annealing and Particle Swarm Optimization because their search paths depend heavily on random choices. Tabu Search may also require repeated trials when it uses random initialization, neighborhood sampling, or tie-breaking.

Multi-objective studies commonly use hypervolume, generational distance, inverse generational distance, spacing, the number of non-dominated solutions, and set-coverage measures. These metrics assess not only convergence toward the Pareto front but also the diversity and distribution of trade-off solutions.

Feature-selection studies use classification accuracy, F-score, precision, recall, Hamming loss, and Normalized Discounted Cumulative Gain. The selected metric should match the predictive task and should be evaluated on data not used to guide the optimization process. The number or percentage of selected features should also be reported so that improvements in predictive performance are not achieved solely by retaining nearly every variable.

Several studies used statistical procedures such as Wilcoxon rank-sum tests, paired t-tests, analysis of variance, and Tukey Honest Significant Difference tests. Statistical testing helps determine whether observed differences are likely to reflect genuine performance distinctions rather than random variation. Nevertheless, the assumptions of each statistical test should be checked, and nonparametric alternatives may be more appropriate when optimization results are not normally distributed.

2.11 Issues and Limitations in Existing Literature

Parameter sensitivity is a recurring issue across all three algorithms. Simulated Annealing requires decisions concerning the starting temperature, cooling factor, transitions per temperature, objective scale, and stopping criteria. Tabu Search requires a tabu tenure, candidate-list size, neighborhood structure, aspiration rule, and stopping condition. Particle Swarm Optimization requires a population size, inertia weight, cognitive and social coefficients, topology, velocity control, and termination rule. Parameter values that perform well for one problem may perform poorly for another.

Many studies select parameters heuristically or copy values from earlier literature without systematic tuning. This makes it difficult to determine whether poor performance results from the algorithm or from unsuitable parameter selection. Excessive tuning on the same test instances may also produce optimistic results that do not generalize to unseen cases.

Experimental conditions are often inconsistent across studies. Algorithms may be implemented using different datasets, hardware, programming languages, computational budgets, stopping rules, and levels of optimization. Some studies compare a newly implemented algorithm with numerical results copied from earlier papers instead of rerunning all methods in the same environment. These differences make it difficult to attribute performance gaps to the algorithms themselves.

Repeated-run reporting is also frequently inadequate. Some studies report only one run, one best result, or a mean without standard deviation. This is insufficient for stochastic optimization because one unusually favorable run may create a misleading impression of performance. Reliable evaluation requires several independent trials and measures of variation.

Several reviewed studies were conducted on small or synthetic problems. Grabusts et al. (2019) used only eight route locations, which is small enough for exact enumeration. Zhang and Sun (2002) used synthetic variable-size feature-selection problems containing 30 to 100 features, which are modest relative to modern datasets containing thousands of variables. Other studies used randomly generated scheduling instances or synthetic error models. Such experiments provide controlled conditions but may not represent the uncertainty, dynamic conditions, and operational constraints of real applications.

Weak or inappropriate baselines are another concern. Some studies compared a proposed method only with its immediate predecessor, random selection, or previously published results. Niño et al. (2012) constructed the reference Pareto front from the outputs of the same two algorithms being compared, weakening the interpretation of its perfect recovery results. Allvi et al. (2020) primarily compared Simulated Annealing with random feature selection rather than stronger established feature-selection methods.

Some papers omitted critical implementation details such as random seeds, exact initial and final temperatures, complete cooling schedules, neighborhood-sampling procedures, archive limits, and precise stopping conditions. Missing details reduce reproducibility and make it difficult to determine whether performance differences reflect algorithmic behavior or implementation choices.

Hybrid methods present an additional interpretive challenge. They frequently outperform canonical algorithms but may contain many interacting components, such as mutation, crossover, exemplar breeding, adaptive leaders, local search, chaotic initialization, and redistribution mechanisms. When a hybrid improves performance, it may be unclear which mechanism was responsible. Increased complexity also creates more parameters and a greater risk of overfitting the algorithm to the selected benchmark set.

2.12 Synthesis of the Reviewed Literature

The reviewed studies establish that Simulated Annealing, Tabu Search, and Particle Swarm Optimization are all capable of solving difficult optimization problems, but they rely on substantially different search mechanisms.

Simulated Annealing maintains one current solution and uses temperature-dependent acceptance to control its movement. Its major strength is its ability to cross inferior regions and escape local optima. Its major weakness is its sensitivity to cooling, cost scaling, and stochastic wandering. A carefully designed neighborhood and temperature schedule can make the method competitive on large problems, while poor calibration can cause excessive computation in low-quality regions.

Tabu Search also maintains one current solution, but it uses explicit memory and best-admissible neighborhood movement. It frequently demonstrates strong local exploitation and performs well on scheduling, routing, and feature-selection problems. Its effectiveness

depends on meaningful neighborhood operators, appropriate tabu tenure, and sufficiently informative candidate evaluation. Excessive restrictions or poorly selected tabu attributes may weaken performance.

Particle Swarm Optimization maintains a population of candidate solutions and uses personal and social learning. Its population can examine several regions at the same time, and its simple information-sharing mechanism can produce rapid convergence. However, the swarm may lose diversity and converge prematurely around a local optimum. Discrete problems also require specialized representations that may substantially alter the original continuous PSO mechanism.

Across direct comparisons, no algorithm consistently dominates every problem. Tabu Search frequently performs strongly on neighborhood-intensive combinatorial problems. Particle Swarm Optimization often provides efficient convergence and scalability when its discrete representation is appropriate. Simulated Annealing can produce competitive solutions but may require more evaluations and greater care in parameter calibration. Hybrid algorithms may combine their complementary strengths, but they also increase complexity and make causal interpretation more difficult.

The relative performance of the algorithms depends on problem type, problem size, encoding, neighborhood or movement design, parameter values, evaluation budget, stopping conditions, and implementation quality. One algorithm should therefore not be declared universally superior based on isolated experiments. A controlled comparison should examine both final solution quality and the process by which that quality is obtained.

2.13 Research Gap

Previous studies have applied Simulated Annealing, Tabu Search, and Particle Swarm Optimization to route optimization, scheduling, and feature selection. However, most studies focus on one application domain and use highly problem-specific implementations. Direct comparison remains difficult because prior research frequently uses different solution representations, objective functions, datasets, parameter settings, computational budgets, stopping criteria, hardware environments, and statistical procedures.

Some comparisons measure runtime, while others use iteration count or objective evaluations. Several studies do not report repeated-run statistics or formal significance tests. Others compare heavily modified or hybrid algorithms with basic implementations, making it difficult to determine whether observed improvements result from the fundamental algorithm or from additional operators.

The three selected problem categories also have substantially different structures. Route optimization commonly uses permutations of locations. Scheduling combines sequencing, resource assignment, precedence, and capacity constraints. Feature selection uses binary subsets and may require an expensive predictive-model evaluation for every candidate. These

differences may favor different search mechanisms. Tabu Search may benefit from clearly defined discrete neighborhoods, Particle Swarm Optimization may benefit from population diversity, and Simulated Annealing may benefit from flexible stochastic movement. Existing findings have not been consolidated sufficiently under one controlled evaluation framework.

The present study addresses this gap by implementing and comparing Simulated Annealing, Tabu Search, and Particle Swarm Optimization across Traveling Salesman Problem route optimization, machine-job scheduling, and machine-learning feature selection. The comparison will use clearly documented parameters, common or comparable computational budgets, multiple independent runs, solution-quality measures, efficiency measures, convergence analysis, and statistical reporting.

The purpose is not to identify one universal winner. Rather, the study seeks to determine which algorithm performs most effectively under specific problem structures and experimental conditions. It also seeks to identify the practical trade-offs associated with each approach, including implementation complexity, parameter sensitivity, scalability, and consistency.

2.14 Significance of the Present Study

The study has theoretical, methodological, and practical significance. From a theoretical perspective, it compares three fundamentally different metaheuristic strategies: probabilistic single-solution search, memory-based neighborhood search, and population-based social search. Evaluating these approaches across permutation, scheduling, and binary-subset spaces may clarify how their search mechanisms respond to different forms of combinatorial structure.

From a methodological perspective, the study promotes more controlled algorithm comparison through transparent implementation, comparable computational limits, repeated trials, statistical analysis, and consistent performance reporting. This approach addresses several weaknesses identified in previous research, including inconsistent budgets, incomplete parameter descriptions, missing variation measures, and comparisons conducted under different experimental environments.

From a practical perspective, the findings may support algorithm selection across several domains. Route-planning practitioners may prioritize distance reduction, runtime, and route stability. Production and manufacturing planners may prioritize makespan, machine utilization, tardiness, and schedule feasibility. Machine-learning practitioners may prioritize predictive performance, dimensionality reduction, and training cost.

By comparing the algorithms across several problem structures, the study may help researchers and practitioners choose a suitable metaheuristic according to the characteristics and requirements of the problem rather than assuming that one algorithm is universally superior.